

Exploration Rovers

Presented by GROUP 5

- NANA KWAME ADOMAKO DAPAAH
- CALEB ALHASSAN
- ELORM AKU ADDO
- ADUO KELVIT
- MICHELLE AWURABENA ACQUAH
- TYRONE NII DROMO TETTEY ADDY
- SAMUEL AGYEMANG



TABLE OF CONTENT

- PROBLEM STATEMENT
- METHODOLOGY
- DEMONSTRATION



PROBLEM STATEMENT

- **PREPARATIONS FOR HUMAN MISSIONS**

Rovers are used to understand the environment's topography before sending human beings to explore

- **EXPLORATION OF HARSH ENVIRONMENTS**

It is used to explore places deemed too toxic and harmful for human exploration

PROBLEM STATEMENT

- **EXPLORATION OF CAVES AND CRACKS**

Rovers assist in the exploration of places human beings would not be able to fit or enter or is in danger may face structural damage

- **MOBILITY OVER LARGE AREAS**

Since rovers can move around, it makes it easy to study different terrains and places

- **REMOTE OPERATIONS**

Makes it easy to explore places far away from the comfort of a lab or any facility

COMPONENTS USED

SERVO MOTORS

PWM MOTOR CONTROLLER

LIPO BATTERY

ESP 32

ULTRASONIC SENSOR

HOW THE SYSTEM WORKS

The two motors move with varying speeds mimicking turns. The main idea is to traverse the entire terrain. Think of it like a ripple in water. It starts from the center and moves till it gets to the ends of the body of water. The robot does the same. The varying speeds produce an imbalance in the forces acting on the robot. The robot in turn, swings in that direction with the lower enable values{value given in the program to determine the speed}. As the robot moves in a spiral manner, it ends up moving across the entire terrain



CODE

```
const int ENA = 14;
const int ENB = 32;
const int IN1 = 26;
const int IN2 = 27;
const int IN3 = 25;
const int IN4 = 33;
const int TRIG = 5;
const int ECHO = 18;

const int CRUISE_SPEED = 250;
const int MANEUVER_SPEED = 220;
const float OBSTACLE_CM = 28.0f;
const float CLEAR_CM = 38.0f;
const unsigned long QUICK_DOUBLE_MS = 2000;
const unsigned long CLEAR_RESET_MS = 1500;
const unsigned long TURN_RIGHT_MS = 700;
const unsigned long BACK_UP_MS = 950;
const unsigned long TURN_LEFT_MS = 700;

enum Mode : uint8_t { CRUISE, TURN_RIGHT, BACK_UP, TURN_LEFT };

Mode mode = CRUISE;
unsigned long maneuverEndMs = 0;
unsigned long lastObstacleEventMs = 0;
static unsigned long clearPathSinceMs = 0;
```

```
float readDistanceCm() {
    digitalWrite(TRIG, LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG, LOW);
    unsigned long duration = pulseIn(ECHO, HIGH, 30000UL);
    if (duration == 0) {
        return 400.0f;
    }
    return (duration * 0.0343f) / 2.0f;
}

void motorsStop() {
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, LOW);
    digitalWrite(IN3, LOW);
    digitalWrite(IN4, LOW);
    analogWrite(ENA, 0);
    analogWrite(ENB, 0);
}

void motorsForward(int speedPWM) {
    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);
    digitalWrite(IN3, HIGH);
    digitalWrite(IN4, LOW);
    analogWrite(ENA, speedPWM);
    analogWrite(ENB, speedPWM);
}
```

```
void motorsBack(int speedPWM) {
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, HIGH);
    digitalWrite(IN3, LOW);
    digitalWrite(IN4, HIGH);
    analogWrite(ENA, speedPWM);
    analogWrite(ENB, speedPWM);
}

void motorsPivotRight(int speedPWM) {
    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);
    digitalWrite(IN3, LOW);
    digitalWrite(IN4, HIGH);
    analogWrite(ENA, speedPWM);
    analogWrite(ENB, speedPWM);
}

void motorsPivotLeft(int speedPWM) {
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, HIGH);
    digitalWrite(IN3, HIGH);
    digitalWrite(IN4, LOW);
    analogWrite(ENA, speedPWM);
    analogWrite(ENB, speedPWM);
}
```

```

void setup() {
  pinMode(ENA, OUTPUT);
  pinMode(ENB, OUTPUT);
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);
  pinMode(TRIG, OUTPUT);
  pinMode(ECHO, INPUT);
  clearPathSinceMs = millis();
}

void loop() {
  const unsigned long now = millis();
  float cm = readDistanceCm();
  const bool obstacle = (cm > 0.0f && cm < OBSTACLE_CM);
  const bool pathClear = (cm >= CLEAR_CM);

  if (pathClear) {
    if (clearPathSinceMs == 0) {
      clearPathSinceMs = now;
    }
    if (now - clearPathSinceMs >= CLEAR_RESET_MS) {
      lastObstacleEventMs = 0;
    }
  } else {
    clearPathSinceMs = 0;
  }
}

```

```

switch (mode) {
  case CRUISE:
    motorsForward(CRUISE_SPEED);
    if (obstacle) {
      const bool doubleHit =
        (lastObstacleEventMs != 0) &&
        ((now - lastObstacleEventMs) <= QUICK_DOUBLE_MS);
      if (doubleHit) {
        lastObstacleEventMs = 0;
        mode = BACK_UP;
        maneuverEndMs = now + BACK_UP_MS;
        motorsBack(MANEUVER_SPEED);
      } else {
        lastObstacleEventMs = now;
        mode = TURN_RIGHT;
        maneuverEndMs = now + TURN_RIGHT_MS;
        motorsPivotRight(MANEUVER_SPEED);
      }
    }
    break;
  case TURN_RIGHT:
    motorsPivotRight(MANEUVER_SPEED);
    if (now >= maneuverEndMs) {
      mode = CRUISE;
      motorsStop();
    }
    break;
  case BACK_UP:
    motorsBack(MANEUVER_SPEED);
    if (now >= maneuverEndMs) {
      mode = TURN_LEFT;
      maneuverEndMs = now + TURN_LEFT_MS;
      motorsPivotLeft(MANEUVER_SPEED);
    }
    break;
}

```

```
case TURN_LEFT:
    motorsPivotLeft(MANEUVER_SPEED);
    if (now >= maneuverEndMs) {
        mode = CRUISE;
        motorsStop();
        lastObstacleEventMs = 0;
    }
    break;
```

END OF CODE